

Agentic Engineer

¿La IA te acelera... hasta que revisas el PR?
Spec-Driven Development + IA para recuperar el control en tu equipo de desarrollo

USA LAS FLECHAS PARA DESPLAZARTE





Índice

01. Cómo usar este playbook	05
02. El problema: la IA te acelera...hasta que revisas el PR	06
Los datos que lo confirman	07
03. La causa: el trabajo mal definido	08
Cómo identificar los síntomas	07
04. Los 3 pilares del uso efectivo de la IA	08
Pilar 1: Tool. La herramienta y su configuración	11
Pilar 2: Prompt. Cómo le das instrucciones a la IA	13
Pilar 3: Context. El escalón que la mayoría se salta	17
05. Context Engineering: la receta perfecta del contexto	18
Especificaciones técnicas	18
Flujo de trabajo	19
Contexto compartido y actualizado	19
06. Qué es Spec-Driven Development	21
Qué incluye una buena spec de feature	23
El impacto: spec mala vs spec bien escrita	23
Cuando el contrato cambia: gestión del cambio en SDD	24
Del SDLC clásico al SDLC automatizado	25

07. OpenSpec en la práctica: del framework a la demo técnica	27
El ciclo en tres pasos: Proposal → Apply → Archive	27
El ciclo no es lineal: cómo gestionar la iteración	27
De requisito a tarea técnica detallada	28
Ciclo de desarrollo OpenSpec LIDR	29
La granularidad correcta de las tareas	30
/code-review: qué hace realmente el agente y cuándo no es suficiente	30
La estructura de ficheros y cómo escala	31
Lo que genera el ciclo completo automatizado	32
Cuatro prácticas que lo hacen funcionar	32
Frameworks SDD: por dónde empezar	33
08. Casos de éxito. Adopción IA en equipos de desarrollo	34
Mercado Libre · E-commerce y fintech · América Latina · +18 países	34
Bdeo · Soluciones tech para seguros y movilidad · Scaleup	34
InfoJobs (Adevinta) · Plataforma de empleo · +11M candidatos · Enterprise	35
09. AI Assessment 2026: descubre tu nivel	36
10. ¿Quieres convertirte en Agentic Engineer?	37
Para quién es AI4Devs	37
Qué incluye	37
Mentores	38
Ya confían en nosotros	38
Lo que dicen los developers que ya pasaron por el programa	39
Garantía de devolución y facilidades de pago	40
Proceso de inscripción	40



Alvaro Moya 🏆
@LIDR.co

"El impacto más relevante de la IA en desarrollo no es ni mucho menos programar más rápido. El coste de hacer buen código ha colapsado, ya no hay excusa para aumentar la calidad, desde buenas prácticas y documentación exhaustiva a tests unitarios y E2E automáticos. Ahí está el verdadero impacto en productividad a largo plazo."

— Álvaro Moya, Fundador de LIDR.co

01. Cómo usar este playbook

Este playbook no está pensado para leerse de una vez, sino para volver a él en cada avance de nivel. Los primeros capítulos explican el problema y la causa de una adopción IA inadecuada. Los centrales te dan el framework y la metodología. Y los últimos son recursos accionables: casos de éxito, evaluación de tu nivel y el Máster AI4Devs para quien quiera ir más lejos.

El mapa del playbook

1	Cómo se usa	Instrucciones básicas de navegación
2	El problema	Por qué la IA individual no escala y dónde se pierde el impacto real.
3	La causa	El cuello de botella es la falta de definición.
4	Los 3 pilares	Tool, Prompt y Context Engineering: el triángulo que lo sostiene todo.
5	Context Engineering	Cómo construir el contexto que hace que la IA no empiece desde cero.
6	Qué es SDD	Spec-Driven Development: la especificación como contrato ejecutable.
7	OpenSpec en la práctica	El framework LIDR, los comandos del ciclo completo y la demo técnica.
8	Casos de éxito	Mercado Libre, Bdeo e InfoJobs: adopción real con resultados medibles.
9	AI Assessment	Evalúa tu nivel real de adopción de IA en 6 dimensiones. Gratis.
10	Agentic Engineer	El método probado para adoptar el nuevo paradigma.

Elige tu ruta de lectura

Quieres resultados rápidos

Empieza por el Cap 07 y la demo técnica.
Luego vuelve al 04 y 05 para entender el porqué.

→ Cap 07 → 04 → 05 → 09

Quieres entender desde la base

Lee en orden del 02 al 07. Es el arco completo: problema, causa, solución, implementación.

→ Cap 02 → 03 → 04 → 05 → 06 → 07

Lideras un equipo de desarrollo

Empieza por los casos de éxito para ver el impacto real. Luego el framework y el assessment.

Quieres medir dónde estás ahora

Ve directamente al AI Assessment.
Luego usa los resultados para priorizar qué leer.

→ Cap 09 → Según resultados



Una última cosa antes de empezar: El Cap 07 tiene una demo técnica en vivo y acceso al repositorio de OpenSpec. Son los recursos más accionables del playbook. No los dejes para el final.

02. El problema: la IA te acelera... hasta que revisas el PR

Imagina la escena. Lunes, 9 de la mañana. Abres el IDE, le pides a tu copiloto que implemente la nueva feature, y en 12 minutos tienes el código. ¡Genial! Velocidad. La promesa de la IA cumplida.

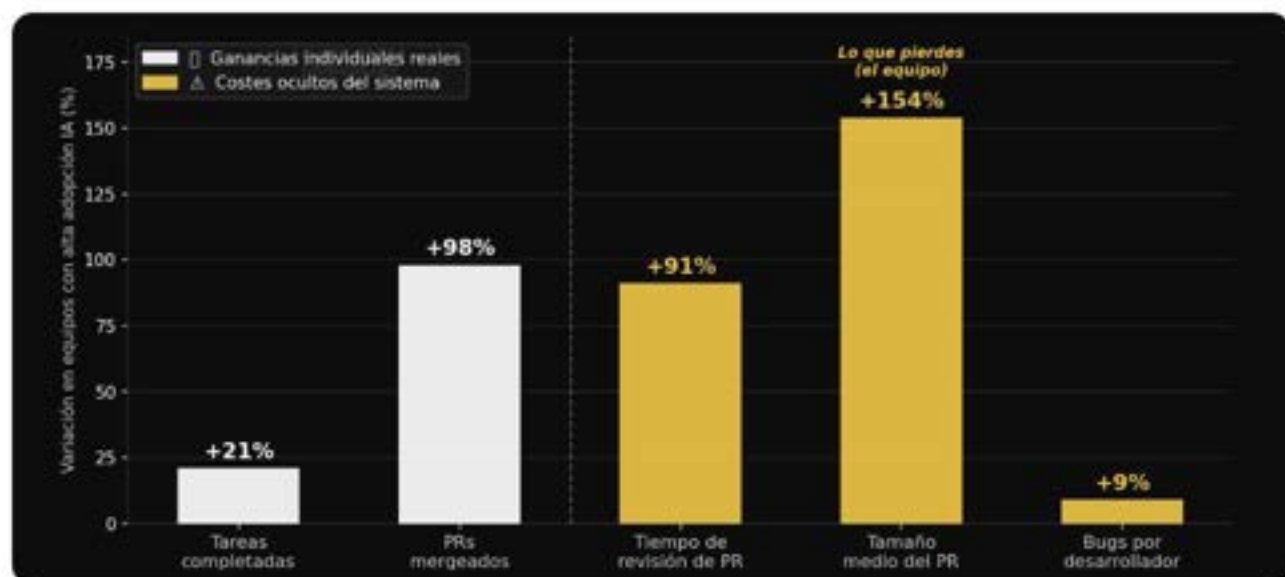
Luego llega el miércoles. El PR lleva 2 días en revisión. Tu compañero senior está encontrando edge cases que el código no cubre. QA ha rechazado 3 veces la misma funcionalidad porque no cumple con los criterios de aceptación que nadie había escrito en ningún sitio. Y tú, mientras tanto, estás reescribiendo lo que la IA generó en 12 minutos... durante 3 días.

¿Te suena? No es un problema de la IA. Es un problema de método.

¿Cómo podemos resolverlo eficientemente?

El patrón se repite en equipos de todo tipo y tamaño. La IA genera código más rápido que nunca. Pero si la entrada es vaga, el output también lo es. El coste de corrección no desaparece: se desplaza. Se mueve del momento de escritura al de revisión, validación, QA y producción.

La trampa es sutil: cuando empiezas a usar un copiloto, la sensación inmediata es de velocidad. Escribes menos, produces más. Lo que no ves es la deuda que se acumula aguas abajo. La IA es demasiado buena siguiendo instrucciones ambiguas: genera código coherente, sintácticamente correcto, aparentemente funcional... que no hace lo que necesitabas.



La Paradoja de Productividad IA

La Paradoja de Productividad IA · Fuente: Faros AI, AI Productivity Paradox Report 2025



En 2025, las empresas invirtieron entre 30 y 40 mil millones de dólares en IA generativa. El 95% de ellas reportó cero retorno medible. La tecnología no falla, es que se usa sin estructura.

- El informe [AI Productivity Paradox](#) de Faros AI (2025) analizó la telemetría de más de 10.000 developers en 1.255 equipos. Los resultados son difíciles de ignorar. El cuello de botella no desaparece: se desplaza de quien escribe código a quien lo revisa. Y el equipo entero lo paga.
- Y no es solo el review. La adopción de IA está asociada con un aumento del 9% en bugs por developer y un incremento del 154% en el tamaño medio de los PRs. Más código, más grande, más complejo, con más errores. Llegando al revisor más rápido que nunca.
- La consecuencia es la que describe la Ley de Amdahl: acelerar solo la escritura de código no ayuda si las reviews y los tests no mantienen el mismo ritmo. El valor creado por la IA se absorbe en los cuellos de botella posteriores según [Index.dev](#)
- El problema de calidad lo confirma Stack Overflow: el 66% de los developers reporta que su mayor frustración con las herramientas de IA es que las soluciones son "casi correctas, pero no del todo". Y el 45% pierde tiempo significativo debuggeando código generado por IA ([The AI Productivity Paradox Report](#), Faros).
- El dato que más debería hacer reflexionar a cualquier equipo viene del informe DORA de Google 2024: a medida que la adopción de IA

aumentaba entre equipos, el throughput de entrega caía un 1,5% y la estabilidad de entrega se reducía un 7,2%. Los developers individuales iban más rápido. Los equipos, más despacio.

- Y quizás el más incómodo de todos: un ensayo controlado aleatorio del laboratorio independiente METR, publicado en julio de 2025, midió el impacto real de las herramientas de IA en 16 developers experimentados completando 246 tareas reales en sus propios repositorios. Los developers tardaron un 19% más cuando usaban herramientas de IA que cuando trabajaban sin ellas. La IA los hacía más lentos. Lo más llamativo: antes del estudio, los developers predijeron que la IA reduciría su tiempo un 24%. Al terminar, seguían creyendo que les había hecho un 20% más rápidos. La realidad era exactamente la opuesta.

Esto no significa que la IA no funcione. Significa que la IA sin método no funciona como equipo. Las ganancias individuales son reales. El problema es que el sistema entero —review, QA, integración, producción— no está preparado para absorberlas.

🔑 La pregunta clave

¿Qué pasaría si antes de darle trabajo a la IA, le dieras también el contexto completo de lo que necesitas, cómo tiene que encajar en el sistema existente y cuáles son los criterios exactos de éxito? Eso es exactamente lo que resuelve Spec-Driven Development.

03. La causa: el trabajo mal definido

Hay un problema que lleva décadas en los equipos de desarrollo y que la IA no solo no resuelve, sino que amplifica: **el trabajo llega mal definido a quien lo tiene que ejecutar.**



Todos estos son ejemplos de trabajo que llega al desarrollador sin especificar qué, por qué, cómo ni cuándo es suficiente.

En el mundo pre-IA, ese trabajo mal definido generaba fricción, sí. Pero tenía un freno natural: la velocidad de escritura humana.

El freno que desaparece con IA



El freno que desaparece con IA

Cuando el trabajo llega mal definido al copiloto de IA, los síntomas son siempre los mismos. Reconocerlos es el primer paso para corregirlos.

PRs con comentarios "esto no era lo que pedí"	Developer hace suposiciones e implementa todas (o ninguna)
La misma feature implementada dos veces de forma distinta	Code review dura días porque nadie sabe qué se intenta hacer
QA rechaza tickets con requisitos no documentados	Deuda técnica que crece por decisiones sin información suficiente

Los datos del sistema roto

Síntoma	Dato
PRs con rework o rechazados	Tasa de aceptación del 32,7% vs. PRs manuales — el 67%+ requiere correcciones
IA sin contexto del proyecto	El 65% de developers dice que la IA pierde el contexto relevante en tareas críticas: refactoring, tests o code review
Inconsistencia con estándares del equipo	El 44% dice que la IA degrada la calidad del código y atribuye el problema a gaps de contexto
PRs bloqueados en revisión	Ciclo medio de 7 días — los PRs permanecen 5 días inactivos. Los PRs de IA esperan 4,6x más antes de ser recogidos

Síntomas vs datos

Fuentes: LinearB Software Engineering Benchmarks 2025 - Qodo State of AI Code Quality 2025

El problema clásico del trabajo mal definido se convierte, con IA, en un problema de velocidad de acumulación de errores. La aceleración es real. Pero también lo es el coste de corrección.

 La solución empieza aquí

No se trata de frenar la velocidad de la IA. Se trata de asegurarse de que la IA corre en la dirección correcta desde el principio. Y para eso necesitas una especificación: un contrato claro entre el negocio, el producto y la implementación técnica.

Pilar 2: Prompt. Cómo le das instrucciones a la IA

El prompt engineering no ha muerto, pero sí ha evolucionado. Ahora hablamos de una forma estructurada y reproducible de comunicarte con la IA que funcione en tu equipo, no solo para ti.

Los buenos prompts tienen tres características (como mínimo) que los malos no tienen:

🎯 OBJETIVO CLARO

Qué tiene que hacer y para qué. Sin ambigüedad.

🚫 RESTRICCIONES

Qué NO tocar, qué estándares respetar.

📄 FORMATO DE SALIDA

Código, tests, docs, propuesta en markdown...

▶ Contenido en vídeo complementario:

Para profundizar en estos dos primeros pilares, te sugerimos este vídeo de nuestro canal de YouTube: <https://www.youtube.com/watch?v=eca3IWJgRmA>
Aprenderás a:

1. Elegir adecuadamente tu copiloto y configurarlo para sacarle todo el partido: MCPs, settings, modelo LLM, memoria, etc.
2. Estructurar prompts usando las mejores prácticas y técnicas avanzadas para mejorar la precisión de tus asistentes de código.



04. Los 3 pilares del uso efectivo de la IA

Cuando hablamos de usar la IA de forma efectiva en desarrollo de software, hay tres dimensiones que determinan el 99% de los resultados.

Tres pilares que, cuando se alinean, convierten a cualquier copiloto en una herramienta que multiplica tu capacidad.

- **Tool:** no todas las herramientas son iguales, y controlar bien cada parámetro es la clave.
- **Prompt:** dado que a través de nuestras instrucciones es como operamos a los copilotos IA, es fundamental hacerlo bien.
- **Context:** el pilar más importante a día de hoy para asegurar que lo que genera la IA se alinea con tu forma de programar y con las directrices del proyecto.



TOOL

¿Qué herramienta usas y cómo la configuras?



PROMPT

¿Cómo le das las instrucciones correctas a la IA?



CONTEXT

¿Qué información tiene la IA sobre tu proyecto?

Pilar 1: Tool. La herramienta y su configuración

La herramienta importa, pero menos de lo que crees. Cursor, Windsurf, GitHub Copilot, Claude Code... todos son capaces de generar código de calidad. La diferencia está en cómo los configuras, no en cuál eliges.

Una herramienta mal configurada en un equipo cohesionado rinde mucho mejor que la mejor herramienta del mercado sin configuración. ¿Por qué? Porque la configuración es donde defines el modelo de lenguaje a usar, las reglas del proyecto, los standards de código, las convenciones del equipo, los MCPs que conectan con tus sistemas...

Un copiloto sin configurar es un junior que no sabe nada de tu empresa, tu arquitectura ni tus estándares. Un copiloto bien configurado es un senior que ya ha pasado un onboarding en tu proyecto antes de escribir una sola línea.

⚙️ Qué configurar como mínimo

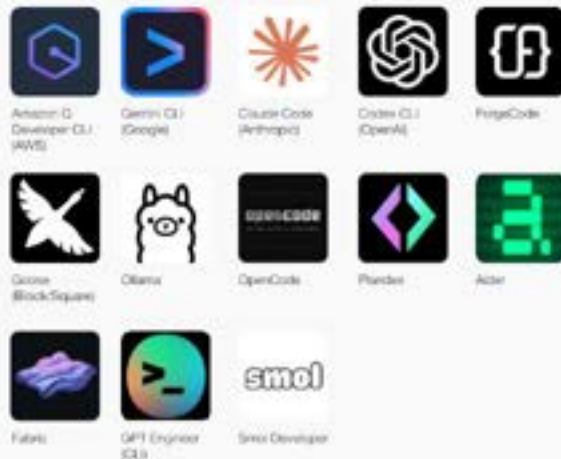
Modelo LLM (no siempre el más potente es el mejor para cada tarea), reglas del proyecto (.cursorrules, CLAUDE.md, .windsurfrules), memoria persistente, MCPs de integración con Jira/Linear, GitHub, bases de datos. Sin esto, tu copiloto trabaja a ciegas.

The AI Coding Landscape in 2026

Tools are everywhere. Knowing when to use each one is the real skill.

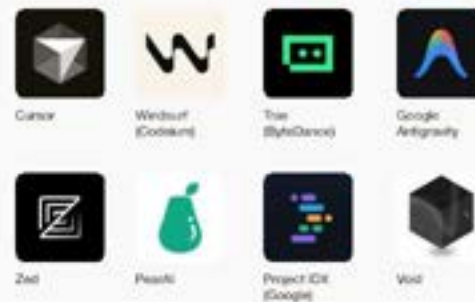
CLI / TERMINAL AGENTS (HEADLESS & ORCHESTRATORS)

Tools that live in your terminal. Ideal for large-scale refactors, scripting, and automation without a graphical interface.



STANDALONE IDES / AGENTIC EDITORS (THE FORKS)

Full development environments where AI isn't a plugin — it's built into the core of the editor.



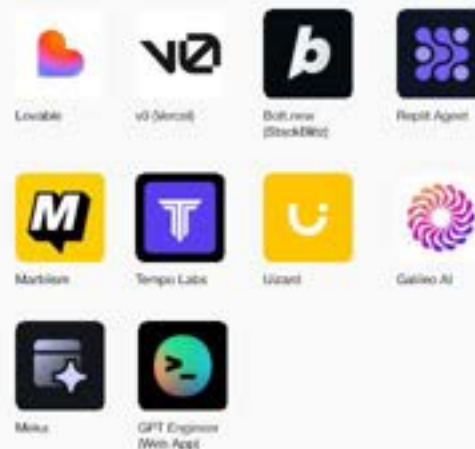
VS CODE / JETBRAINS EXTENSIONS (PLUGINS)

AI integrated into your existing editor. Same environment, new way of working.



WEB · APP GENERATORS (VIBE CODING)

Browser-based platforms that generate complete applications from a single prompt.



Tools change fast. Method is what scales.

**Note: This map focuses on categories, not recommendations.*

Configuración recomendada del IDE

MCPs	Modelos	Otros
Contexto: → Jira (o alternativo) → Context7 → Figma (Framelink) QA: → Playwright Errores & Seguridad: → Sentry → Snyk	→ Opus, Codex o Gemini Pro (últimas versiones) → En Cursor: composer 1 es muy rápido	→ Privacy mode ON → Memories ON

Guía PROMPT ENGINEERING para el desarrollo de software



¿QUÉ ES EL PROMPT ENGINEERING?

Es la disciplina de diseñar y formular instrucciones (prompts) para interactuar de manera efectiva con LLMs. Implica formular preguntas claras y precisas, y entender cómo la estructura del prompt influye en la calidad de las respuestas.



PRINCIPIO BÁSICO

Es la disciplina de diseñar y formular instrucciones (prompts) para interactuar de manera efectiva con LLMs. Implica formular preguntas claras y precisas, y entender cómo la estructura del prompt influye en la calidad de las respuestas.



TIPOS DE PROMPTS

- Zero-shot prompting: Respuesta sin ejemplos previos.
- One-shot prompting: Un ejemplo para orientar la respuesta.
- Few-shot prompting: Múltiples ejemplos para afinar la precisión.



CARACTERÍSTICAS DE UN BUEN PROMPT

Instrucción clara | Estructura lógica y precisión | Contexto adecuado |
 Formato de salida definido | Tono y estilo adecuados | Rol asignado cuando sea relevante



EJEMPLO DE PROMPT

ToDo/Projects tracking system

We are planning the backend for a simple ToDo/Projects tracking system that will be used by multiple users simultaneously. We want to follow DDD for the development of this system, and we also need to follow TDD and BDD.

Prompts avanzados

Técnicas de metaprompting y estructuración para obtener resultados de alta precisión.

META PROMPT · Convierte cualquier prompt básico en uno estructurado de alta precisión

Instructions

You are an expert in prompt engineering.
Given the following prompt, prepare it using best-practice structure (role, objective, etc.) and formatting to achieve a precise and comprehensive result. Stick strictly to the requested objective by carefully analyzing what is asked in the original prompt.

Original Prompt

[Your prompt here, example below] Provide unit tests for the functionality to retrieve the candidates for a position.

Envía un mensaje a ChatGPT



Al enviar un mensaje a ChatGPT, aceptas nuestros [Términos](#) y reconoces que lees nuestra Política de privacidad

ROLE

You are an expert in JavaScript/TypeScript testing specialized in Node.js applications with Express, Prisma ORM, and Jest as the testing framework.

CONTEXT

You are working on an Applicant Tracking System (ATS) that manages candidates, job positions, and interview processes. The system uses:

- **Backend**: Node.js + Express + TypeScript + Prisma ORM + PostgreSQL
- **Testing**: Jest with mocks
- **Architecture**: Controllers → Services → Database

OBJECTIVE

Create comprehensive and exhaustive unit tests for the functionality "retrieve candidates for a position," which includes:

1. **Endpoint**: 'GET /positions/:id/candidates'
2. **Controller**: 'getCandidatesByPosition' in 'positionController.ts'
3. **Service**: 'getCandidatesByPositionService' in 'positionService.ts'

SPECIFIC REQUIREMENTS

Functionality to Test

The 'getCandidatesByPositionService' function must:

- Receive a 'positionId' (number)
- Query 'applications' for that position via Prisma
- Include related data: 'candidate', 'interviews', 'interviewStep'
- Compute the average of interview scores
- Return an array with: 'fullName', 'currentInterviewStep', 'candidateId', 'applicationId', 'averageScore'

Required Test Cases

Successful Tests

- ☒ Return candidates with correct averages when data exists
- ☒ Return an empty array when there are no candidates
- ☒ Compute average 0 when there are no interviews
- ☒ Handle multiple candidates with different scores
- ☒ Correctly format 'fullName' (firstName + lastName)

Error Tests

- ☒ Handle database connection errors
- ☒ Handle unexpected Prisma responses
- ☒ Propagate errors with descriptive messages

Edge Case Tests

- ☒ Handle 'positionId' = 0
- ☒ Handle negative 'positionId'
- ☒ Handle very large 'positionId'
- ☒ Handle interviews with 'null' scores
- ☒ Handle candidate names with special characters

Test Structure

```

...jsx
describe('getCandidatesByPositionService', () => {
  describe('Successful cases', () => {
    // Normal functionality tests
  });
  describe('Error cases', () => {
    // Error handling tests
  });
  describe('Edge cases', () => {
    // Edge case tests
  });
});
...

```

RESPONSE FORMAT

Provide:

1. **Complete test file** with all necessary imports
2. **Mock setup** for Prisma
3. **Realistic test data** for different scenarios
4. **Specific assertions** that validate both returned data and Prisma calls
5. **Explanatory comments** for complex cases

QUALITY STANDARDS

- 100% code coverage
- Independent tests (no interdependencies)
- Realistic ATS-domain test data
- Appropriate mocks without side effects
- Detailed and specific assertions
- Explicit handling of async/await

PREGUNTA AL EXPERTO · Modelado de arquitectura y base de datos con IA

ROLE

You are an expert software engineer with experience in modeling architectures and databases.

OBJECTIVE

Build a data model for an application with the following functionalities:

- Core value: uses AI to generate personalized company reports for investors. It needs the company website URL and some investor data to personalize (ideal round size, preferred industries, etc.).

- The application allows chatting with AI to extract detailed information from each of the reports. Take into account the data modeling required for ChatGPT-like applications and apply best practices for storing conversations.

CLARIFICATIONS ABOUT THE MODEL:

- Users will register with email and password, and we will have Google SSO.
- Profile data will be basic: name, role in the company (text).

- Users will belong to a company that buys seats for its users—a standard SaaS model. This requires having company information and linking it to its seats, as well as assigning a user role (administrator, analyst), configured by the admin.

- Users will pay with Stripe, with an integration similar to ChatGPT. Store the necessary information for this in a 'subscription' table. There is only one price now, but there may be more in the future; create a separate table for pricing.

- Reports will be stored in a 'report' table: the submitted website, the company name, the raw report, and the URL of the generated PDF stored in the cloud for download (not all will generate one).

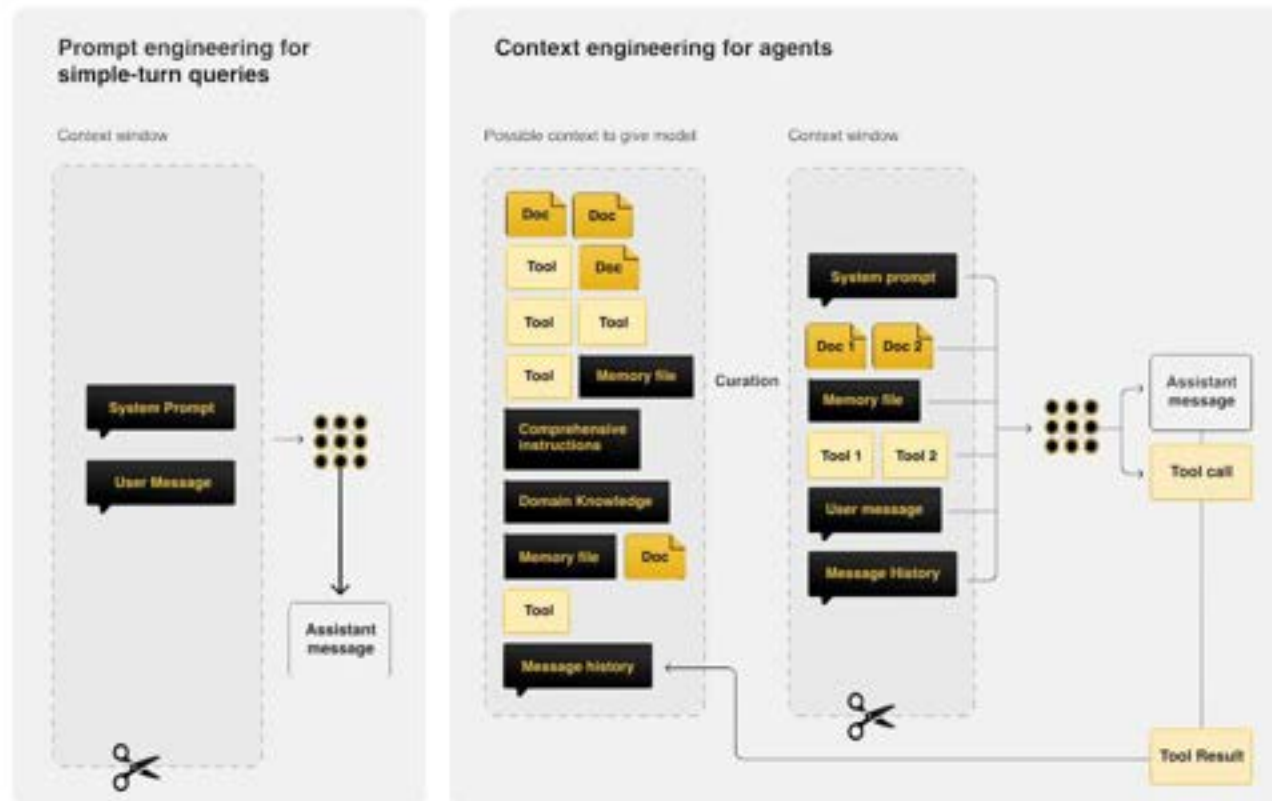
- Chats, as explained, must be stored to maintain history.

Analyze the project and ask me any questions you consider necessary to clarify before proposing the solution.

Pilar 3: Context. El escalón que la mayoría se salta

Es el pilar más importante y el más ignorado. La razón: **el contexto no da feedback inmediato**. Un prompt malo lo ves enseguida. Un contexto pobre es traicionero: el output parece razonable, pero no encaja con tu sistema, no sigue tus convenciones, no tiene en cuenta las decisiones que ya tomaste. El problema aparece más tarde, en el code review, en el QA, en producción.

Prompt engineering VS. context engineering



No se trata de construir un prompt más largo. Se trata de curar, antes de cada sesión, qué información es relevante para la tarea actual y asegurarse de que esa información está disponible en la ventana de contexto del modelo.

Y esa diferencia es la que separa el código que va directo a producción del código que vive eternamente en el PR.

Sin contexto, la IA inventa. Con contexto, la IA extiende.

📌 La clave que lo une todo

La mayoría de equipos trabajan bien en Tool y en Prompt. Donde fallan es en Context. Y el Context Engineering —que veremos en el siguiente capítulo— es precisamente la disciplina de construir ese contexto de forma sistemática y reproducible.

05. Context Engineering: la receta perfecta del contexto

La mayoría de equipos que fracasan con la IA tienen el mismo diagnóstico: **cada developer trabaja con un contexto diferente**. Uno usa Cursor con su propio conjunto de reglas. Otro usa Copilot sin ninguna. El tercero improvisa el prompt en cada sesión. El resultado es código inconsistente, reviews interminables y una IA que parece brillante en los demos y mediocre en producción.

La solución es construir el contexto una sola vez, de forma que toda la organización lo use igual.

La receta perfecta del Contexto

⚙️ Especificaciones técnicas

Cómo es el sistema

Stack, arquitectura, modelo de datos, patrones y convenciones. Todo lo que la IA necesita saber antes de escribir una línea.

🔄 Flujo de trabajo

Cómo funciona el equipo

Proceso completo de ticket a producción: quién hace qué, qué produce, qué define que ese entregable es válido.

🔗 Compartido y actualizado

No es un documento. Es un sistema vivo que crece con cada feature.

Especificaciones técnicas

Todo lo que define cómo está construido tu sistema y bajo qué reglas opera. No qué hace — eso es el negocio — sino cómo está hecho.

Qué documentos

- Stack tecnológico - Lenguajes, frameworks, runtime, versiones
- Setup del entorno - Variables, dependencias, configuración local
- Arquitectura - Estructura de capas, módulos, responsabilidades
- Modelo de datos - Entidades, relaciones, constraints
- Convenciones de código - Naming, patrones acordados por el equipo
- Diseño de APIs - REST vs RPC, formatos, versionado, autenticación
- Estructura de tests - Frameworks, mocking, cobertura mínima
- Git workflow - Branching, commits, PR process
- Error handling y logging - Estrategia de errores, formato de logs
- Seguridad - Autenticación, autorización, validación de inputs

Cómo lo entrega la IA

No en Confluence. En ficheros que el copiloto lee directamente:
 CLAUDE.md Contexto general del proyecto
 .cursorrules Reglas de Cursor
 testing-standards.md Convenciones de tests
 frontend-standards.md Estándares de frontend

Uno por dominio. Todos referenciados desde un archivo base.

Resultado:

La IA ya sabe cómo trabaja tu equipo antes de que le hagas la primera pregunta.

Flujo de trabajo

Las especificaciones técnicas le dicen a la IA cómo es el sistema. El flujo de trabajo le dice cómo funciona el equipo — y qué define que cada entregable es excelente.

Responsable	Entregable	Qué debe contener	Valor con SDD
Negocio / PM	User story	PRD con objetivo, usuarios afectados y valor esperado	Evita features que nadie pidió exactamente
Tech Lead	Ticket técnico	Especificación con criterios de aceptación, restricciones y tareas	La IA genera el 80% si la user story es buena
Developer	Código + tests	Implementación que cumple la spec, con cobertura de casos de error	Con SDD el copiloto ejecuta el contrato
QA automático	Testing report	Suite completa: unit, integration, E2E, análisis de seguridad	Se genera junto al código, no después
Code review	PR validado	Review contra spec, no contra intuición del revisor	Elimina el '¿esto era lo que pedías?'

 Práctica recomendada

Usa `git worktrees` para cada ticket. Cada rama tiene su propio contexto activo sin interferir con el resto del equipo.

Contexto compartido y actualizado

Este es el elemento diferencial. Las especificaciones técnicas y el flujo de trabajo pueden existir en cualquier proyecto bien documentado. Esto es lo que convierte el Context Engineering en una ventaja de equipo, no de individual.

El contexto no es un documento que escribes una vez. Es un sistema vivo: cada spec completada con SDD se convierte en contexto para la siguiente feature. Cada decisión de arquitectura documentada evita que la IA la revierta de forma ingenua. Cada patrón registrado hace que el siguiente prompt sea coherente con lo que ya existe.

Contenido en vídeo complementario:

Para profundizar en este bloque, te sugerimos este vídeo de nuestro canal de YouTube: <https://www.youtube.com/watch?v=okYRbetLh7M>



DORA 2024 (Google)

La adopción de IA sin estructura mejora los resultados individuales pero degrada los resultados del equipo. Throughput de entrega: **-1.5%**. Estabilidad de entrega: **-7.2%**. Los developers individuales iban más rápido. Los equipos, más despacio.

Sin contexto compartido		Con Context Engineering
Cada developer usa su propio estilo y reglas	Consistencia	Mismo output independientemente de quién escribe el prompt
La IA propone soluciones que rompen la arquitectura	Coherencia	Nuevas features encajan con decisiones ya tomadas
La calidad depende del seniority del prompt	Independencia del prompt	El contexto compensa instrucciones imprecisas
Un junior genera código fuera de los patrones del equipo	Independencia del seniority	Un junior genera código con patrones de senior desde el día 1

65%

de developers reportan que la IA pierde el contexto relevante en tareas críticas: **refactoring, tests, code review.**

Godo State of AI Code Quality 2025

El flywheel que lo cambia todo

1. Feature completada con SDD → Genera documentación automáticamente
2. Documentación → contexto → Para la siguiente feature
3. Cada iteración → El sistema se vuelve más preciso

Sin contexto compartido, cada sesión empieza desde cero. Con él, cada sesión empieza desde donde lo dejó el equipo.

06. Qué es Spec-Driven Development

Durante décadas, el código ha sido el rey. Las especificaciones eran una estructura provisional que construíamos y descartábamos en cuanto empezaba el trabajo de programar. Un ticket de Jira como punto de partida, la cabeza como fuente de verdad y el código como destino final.

Spec-Driven Development invierte ese orden de forma deliberada.

La spec se convierte en la fuente. El source-of-truth. Todo lo demás —el código, los tests, las migraciones, los endpoints, la documentación— son proyecciones vivas de esa definición. El developer ya no programa cada detalle: diseña el contrato que los agentes deben cumplir.

Es un cambio de mentalidad enorme.

Y encaja exactamente con lo que los agentes de IA de nueva generación — Claude Code, Cursor, Copilot— ya pueden hacer: comprender contratos estructurados, planificar tareas con subagentes especializados, aplicar cambios con diffs y checkpoints automáticos, y validar que lo generado cumple la spec antes de permitir un merge.

Definición operativa

Spec-Driven Development es una metodología de ingeniería donde la especificación escrita es la fuente de verdad de todo el proceso de desarrollo. La spec es el contrato entre el negocio y la implementación. Define qué tiene que hacer el sistema, cómo tiene que comportarse, cuáles son los criterios de aceptación y qué decisiones técnicas se han tomado. Cuando ese contrato existe, la IA tiene suficiente información para generar código, tests y documentación que realmente encajen.

Contenido en vídeo complementario:

Para profundizar en este bloque, te sugerimos este vídeo de nuestro canal de YouTube: <https://www.youtube.com/watch?v=pGcBdY7hgyc>



SDD no es documentación. Es ingeniería de intención.

El malentendido más frecuente es confundir SDD con "escribir más documentación". No es eso. Documentar el código que ya existe es arqueología. Especificar antes de implementar es ingeniería.

La diferencia es radical: cuando especificas antes de implementar, el proceso de escritura de la spec es en sí mismo una herramienta de pensamiento. Te obliga a descubrir los edge cases antes de que estén en producción, a alinear expectativas antes de que el PR esté abierto, a tomar decisiones de diseño antes de que sean costosas de cambiar.

Y cuando esa spec la tiene la IA en su contexto, el output no es código genérico: es código que ya sabe dónde encaja, con tests que cubren los criterios de aceptación que escribiste y documentación que refleja las decisiones que tomaste.

Dos niveles de spec: sistema y feature

Uno de los errores más frecuentes al adoptar SDD es intentar usar el mismo artefacto para dos propósitos radicalmente distintos. Hay dos niveles de especificación que deben existir en paralelo y no deben confundirse:

Tipo	Propósito	Cuándo se escribe	Quién la mantiene
Spec de sistema	Define la arquitectura, convenciones globales, modelo de datos, patrones y decisiones técnicas permanentes del proyecto	Al inicio del proyecto y se actualiza con cada decisión de arquitectura significativa	Tech Lead / CTO
Spec de feature	Define un cambio concreto dentro del sistema: qué se construye, sus criterios de aceptación, sus restricciones y sus tareas	Antes de cada ciclo de desarrollo, por cada feature o historia de usuario	Developer responsable de la feature

La spec de sistema le dice a la IA cómo es el mundo en el que vive el proyecto. La spec de feature le dice a la IA qué tiene que hacer hoy. Ambas deben estar presentes en el contexto del agente en cada sesión.

Cuando solo existe la spec de feature, el agente genera código que hace lo correcto funcionalmente pero que rompe los patrones del equipo o introduce decisiones de arquitectura inconsistentes. Cuando solo existe la spec de sistema, el agente no sabe qué construir ahora. Las dos se necesitan.

Qué incluye una buena spec de feature

Una spec en SDD no es un documento de 50 páginas. Es un documento estructurado, markdown-first, que incluye los elementos esenciales para que tanto un humano como una IA puedan entender exactamente qué se quiere construir y por qué.

Elemento	Qué responde	Ejemplo concreto
Objetivo	¿Por qué lo hacemos?	Reducir el tiempo de revisión de candidatos un 40% eliminando la navegación manual entre perfiles
Capacidades	¿Qué debe hacer el sistema?	El sistema permite filtrar candidatos por estado (activo / descartado / en proceso) y ordenar por score
Criterios de aceptación	¿Cómo sabemos que está bien?	Dado un recruiter con 50 candidatos, cuando filtra por estado 'en proceso', entonces solo ve los candidatos en ese estado y el contador se actualiza
Restricciones	¿Qué no puede hacer?	Sin modificar el modelo de datos existente de candidates. Sin tocar la capa de servicios. Sin afectar el rendimiento en listas de más de 1.000 candidatos
Decisiones técnicas	¿Cómo lo hacemos?	PATCH en lugar de PUT para actualizaciones parciales. Filtrado en cliente para listas < 500, server-side para listas mayores
Casos de error	¿Qué pasa cuando falla?	Si el API devuelve 500, mostrar banner de error no bloqueante. Si la lista está vacía tras filtrar, mostrar estado vacío con CTA para limpiar filtros
Tareas	¿Qué pasos de implementación?	1. Añadir componente FilterBar con tests unitarios. 2. Integrar en CandidateList. 3. Test E2E de flujo completo. 4. Documentar en ADR

El impacto: spec mala vs spec bien escrita

La diferencia entre una spec mediocre y una buena spec no se mide en líneas de texto, sino en el output que produce el agente. Aquí tienes el mismo requisito en dos versiones y lo que genera cada una:

Spec incompleta	Spec bien escrita
Requisito: "Añadir filtros a la lista de candidatos"	Requisito: "Añadir filtro de estado a CandidateList con comportamiento server-side para listas > 500 registros, sin modificar CandidateService"
Lo que genera la IA: Filtros en cliente para todos los casos, sin tests de edge case, estructura propia no alineada con el componente existente	Lo que genera la IA: Lógica condicional cliente/servidor, tests para ambas ramas, código que extiende el componente existente respetando sus patrones
Dónde explota: En code review ("esto no era lo que pedí"), en QA (falla con 600 candidatos), en producción (degradación de rendimiento)	Resultado: PR aprobado en primera revisión. Criterios de aceptación validados automáticamente antes de que llegue al revisor humano

El mismo agente, el mismo modelo, el mismo copiloto. La única variable es el input. Una spec incompleta no produce código malo por accidente: produce el mejor código posible para una pregunta mal formulada. La IA es demasiado buena siguiendo instrucciones imprecisas.

Cuando el contrato cambia: gestión del cambio en SDD

La pregunta que todo dev senior se hace al leer sobre SDD es legítima: ¿qué pasa cuando el contrato cambia a mitad del desarrollo? Porque el scope cambia. El PM llega con nuevos requisitos el miércoles.

El backend descubre que la API de terceros no soporta lo que asumimos. Aparece un bug crítico en producción que redirige las prioridades.

En SDD, el cambio no es un fallo del sistema. Es un ciudadano de primera clase con su propio proceso:

🔑 El principio del cambio en SDD

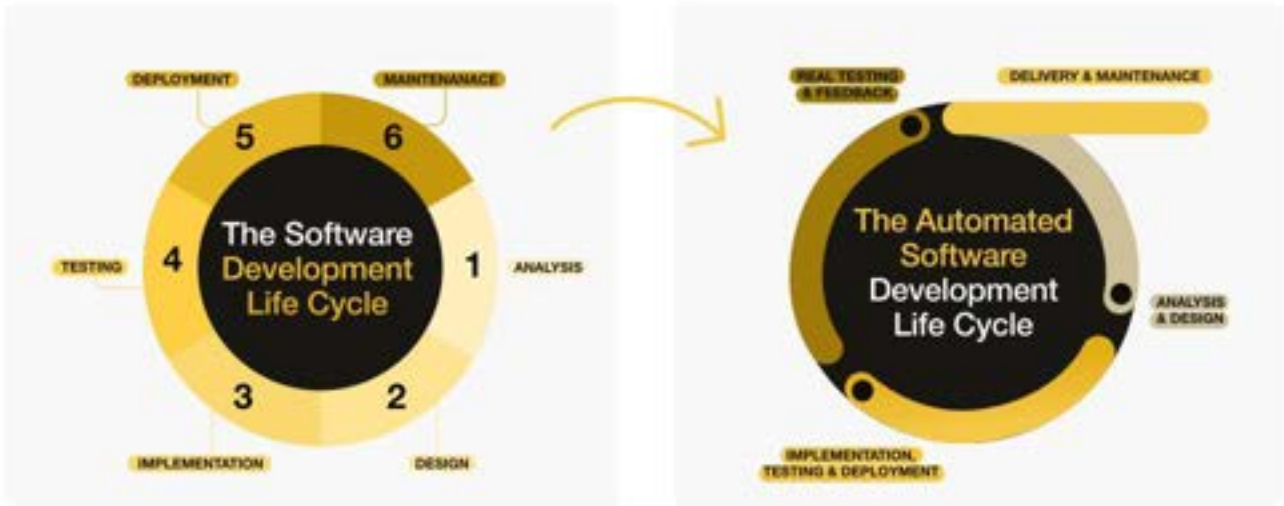
Una spec es inmutable una vez archivada. Los cambios futuros generan nuevas specs. Esta regla es la que garantiza la trazabilidad completa: cada commit puede seguirse hasta el requisito exacto que lo originó, incluso meses después.

Situación	Protocolo SDD	Qué NO hacer
El requisito cambia antes de ejecutar la spec	Volver a /enrich.us con el cambio. El agente actualiza el Proposal antes de ejecutar. Cero código generado sobre una spec obsoleta	Ejecutar /apply con la spec anterior y "arreglar después"
El requisito cambia mientras /apply está en curso	Interrumpir la ejecución. Actualizar la spec de feature. Lanzar /apply de nuevo desde el punto de cambio. Los artefactos ya generados se revisarán en /code-review	Continuar la ejecución y parchear el output manualmente
Se descubre durante el desarrollo que la decisión técnica inicial era incorrecta	Documentar el cambio en la spec (sección 'Decisiones técnicas revisadas'). Actualizar la spec de sistema si afecta a patrones globales. Archivar la decisión descartada con su razonamiento	Cambiar el código sin actualizar la spec, creando divergencia entre intención y realidad
El cambio afecta a features ya archivadas	Abrir una nueva spec de feature para el cambio (nunca editar una spec archivada). La spec archivada documenta el estado histórico; la nueva spec documenta la evolución	Editar specs archivadas, perdiendo la trazabilidad histórica

Del SDLC clásico al SDLC automatizado

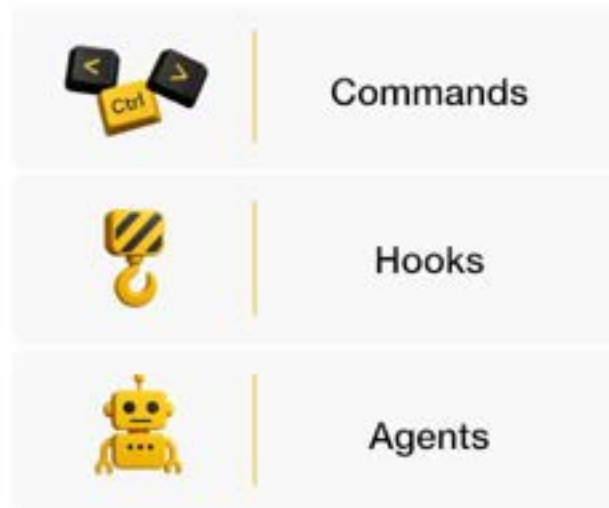
Durante décadas, el SDLC ha sido el marco de referencia: análisis, diseño, implementación, pruebas, despliegue, mantenimiento. Fases claras, responsables definidos, entregables medibles. En papel, impecable. En la práctica, siempre se colaba lo mismo: tickets mal definidos, documentación desactualizada, test cases que nadie mantenía, y el clásico "funciona en mi entorno".

El resultado: equipos apagando incendios, revisiones eternas y un flujo que dependía del héroe del sprint más que del sistema. SDD reinterpreta ese modelo con agentes. No lo reemplaza. Lo actualiza.



En el SDLC automatizado, las fases no desaparecen: se comprimen y se interconectan. El análisis y el diseño ocurren mientras se escribe la spec. La implementación, los tests y el despliegue ocurren cuando el agente ejecuta la spec. El feedback real llega antes porque los criterios de aceptación ya estaban definidos desde el principio.

Y aquí es donde las funcionalidades de los copilotos avanzados nos permiten orquestar el proceso:



- Los **commands** ejecutan acciones concretas (refinar la spec, generar código, crear tests, ejecutar migraciones). Para ello se apoya en agentes especializados.

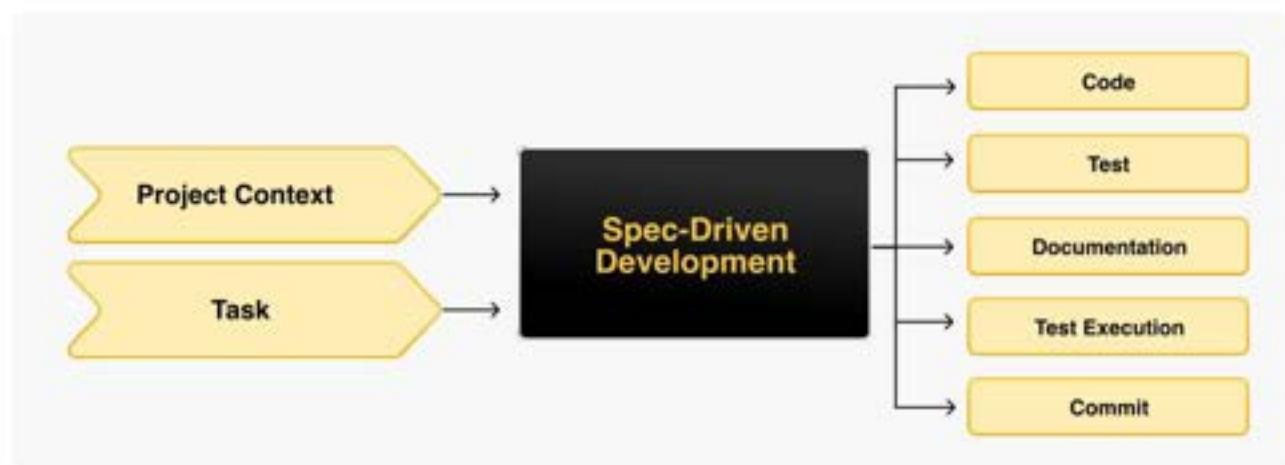
- El agente realiza las tareas basándose en la **spec**, ya sea documentar al detalle lo que se ha de hacer, o programarlo.
- Los **hooks** permiten automatizar acciones en el ciclo de ejecución de Claude (todo lo que sucede desde que recibe el prompt hasta que devuelve el control) de manera sistemática.

Todo fluye, se comprueba y se documenta en tiempo real. No se trata de eliminar a los developers. Se trata de liberarlos: menos tiempo apagando fuegos, más tiempo diseñando sistemas que se autocorrigien.

SDD y el Thoughtworks Technology Radar

En 2025, el Thoughtworks Technology Radar —uno de los índices de referencia del sector tech— incorporó SDD como práctica emergente de alto impacto. No es un experimento de equipo startup. Es una práctica que ya está adoptando la ingeniería enterprise.

Recuerda: en este nuevo paradigma, las especificaciones exhaustivas son esenciales para sacar los mejores resultados de Spec-Driven Development.



07. OpenSpec en la práctica: del framework a la demo técnica

Saber que necesitas una spec es el primer paso. Saber exactamente cómo construirla, dónde guardarla, qué comandos ejecutar y en qué orden es lo que convierte la teoría en un flujo repetible.

OpenSpec es el framework que usamos en LIDR para operacionalizar Spec-Driven Development en proyectos reales. Funciona tanto en proyectos nuevos como en proyectos existentes, se integra con cualquier copiloto —Cursor, Claude Code, Copilot— y estructura el trabajo en torno a un principio simple: la intención primero, la ejecución después.

El ciclo en tres pasos: Proposal → Apply → Archive

Todo el flujo de OpenSpec se reduce a tres fases. La simplicidad es deliberada: lo que no se puede explicar en tres palabras no se adopta en producción.



El ciclo no es lineal: cómo gestionar la iteración

La descripción en tres pasos puede dar la impresión de que el proceso es lineal. En la realidad del desarrollo, no lo es, y OpenSpec lo contempla explícitamente. El flujo tiene puntos de retorno definidos.

Los tres escenarios de iteración más frecuentes son:

- **Descubrimiento durante /apply:** el agente genera código y detecta que la spec no contemplaba un caso de uso del sistema existente. El protocolo correcto es pausar, actualizar la spec y relanzar /apply desde el punto de cambio. No parchear el código y seguir.
- **Feedback del code review humano:** el revisor identifica un problema de diseño. Si el cambio es menor (renombrar, ajustar lógica), se resuelve en el PR. Si el cambio afecta a la intención de la feature, se vuelve a Proposal y se actualiza la spec antes de relanzar.
- **Cambio de requisito externo:** el PM cambia el scope a mitad del ciclo. Ver sección de gestión del cambio del capítulo 6: el proceso es actualizar la spec primero, ejecutar después.

⚠ La regla de oro de la iteración

Nunca hay código que no tenga una spec. Si tienes que hacer un cambio que no está contemplado en la spec activa, actualiza la spec primero. Aunque sea un cambio de dos líneas. La trazabilidad completa vale exactamente ese overhead.

De requisito a tarea técnica detallada

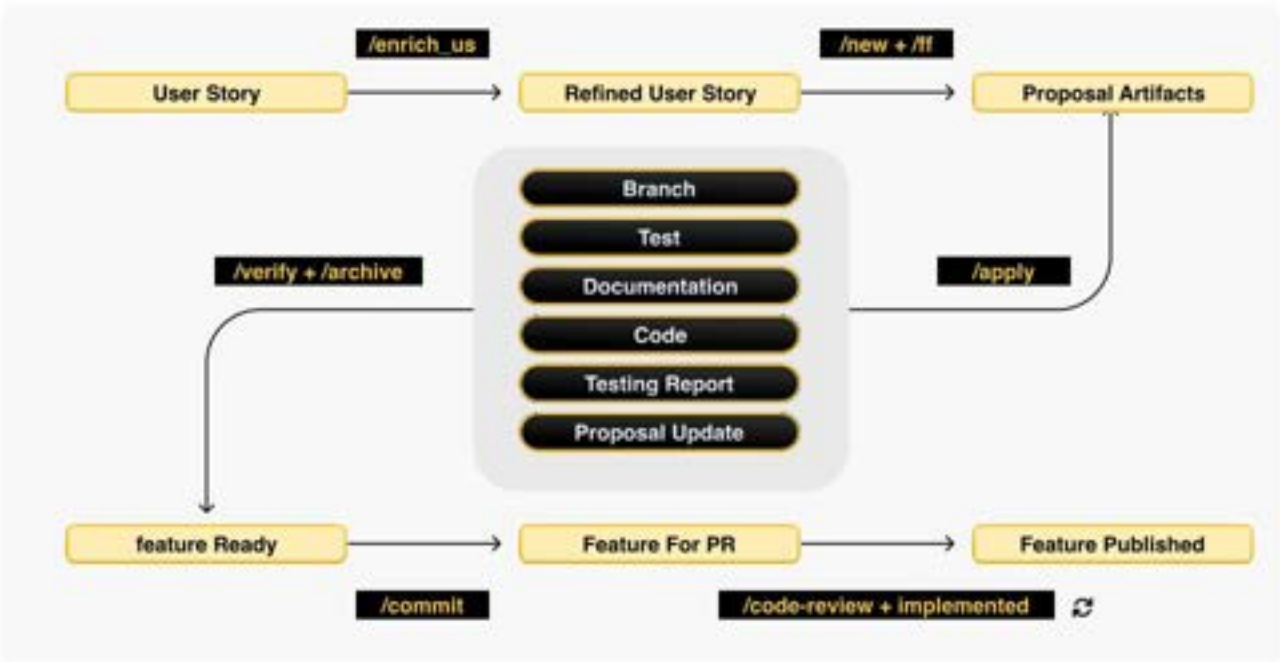
Todo empieza antes de abrir el IDE. El input del ciclo no es un ticket de Jira de una línea — es una historia de usuario convertida en contrato técnico exhaustivo. Cuanto más preciso es el input, más limpio es el output.

📺 Contenido en vídeo complementario:

Para ver exactamente cómo construir esa tarea detallada en la práctica:
<https://www.youtube.com/watch?v=g3pvZSRueug&feature=youtu.be>



Ciclo de desarrollo OpenSpec LIDR



En la práctica, el flujo tiene más granularidad. Estos son los pasos exactos con los comandos que los accionan:

Comando	Fase	Qué hace el sistema
/enrich_us	Enriquecer historia	El agente analiza la user story, detecta ambigüedades, formula las preguntas que nadie había hecho, y produce una Refined User Story con el contexto técnico necesario para especificar bien. Este es el paso más infrutilizado y el más valioso: el 80% de los fallos en SDD ocurren por saltárselo.
/new + /ff	Generar Proposal	A partir de la user story refinada, el agente genera los Proposal Artifacts: spec completa con diseño, tareas y criterios de aceptación. Este es el contrato. Aquí es donde se definen también los casos de error y las restricciones explícitas.
/apply	Ejecutar contrato	El agente ejecuta el contrato. Genera en paralelo: Branch, Tests, Documentation, Code, Testing Report y Proposal Update. Todo alineado con la spec. Importante: el agente no improvisa fuera de la spec. Si encuentra ambigüedad, la señala y espera instrucción.
/code-review	Revisión contra spec	El agente hace una primera revisión del código generado contra la spec, detecta inconsistencias y las resuelve antes de que llegue al revisor humano. Ver sección siguiente para los criterios exactos de esta revisión.
/commit	Commit estructurado	El agente genera el mensaje de commit estructurado (Conventional Commits) con referencia a la spec y mueve el trabajo a Feature Ready.
/verify + /archive	Verificar y archivar	Verificación final contra todos los criterios de aceptación de la spec. Si el contrato se cumple, la proposal se integra en specs/ y la feature pasa a Published. Si no, se abre un ciclo de corrección con referencia explícita al criterio fallido.

La granularidad correcta de las tareas

El problema más frecuente al ejecutar /apply es que el agente improvisa. Y casi siempre la causa es la misma: las tareas en la spec eran demasiado grandes o demasiado vagas. Este es el criterio que marca la diferencia:

Tarea demasiado grande (el agente improvisa)	Tarea bien granularizada (el agente ejecuta)
Implementar el sistema de filtrado de candidatos	Crear componente FilterBar con props: filters (FilterState), onChange (callback). Incluir tests unitarios para: estado vacío, un filtro activo, múltiples filtros activos
Añadir los tests	Test E2E: dado recruiter autenticado en /positions/123/candidates, cuando selecciona filtro 'en proceso', entonces la URL incluye ?status=in_progress y la lista muestra solo candidatos con ese estado
Documentar los cambios	Actualizar ADR-0042 con la decisión de filtrado client-side vs server-side. Añadir ejemplo de uso de FilterBar en el Storybook existente bajo categoría 'Candidates'

La regla práctica: una tarea está bien granularizada cuando puedes escribir su criterio de éxito en una sola frase. Si necesitas "y" o "además" para describir qué debe hacer, divídela.

/code-review: qué hace realmente el agente y cuándo no es suficiente

El /code-review del agente contra la spec es uno de los pasos más poderosos del ciclo y, al mismo tiempo, el más incomprendido. Es importante ser preciso sobre lo que valida y lo que no:

El agente valida	El agente NO puede validar
Que cada criterio de aceptación de la spec tiene un test que lo cubre	Que el criterio de aceptación captura correctamente la necesidad real de negocio
Que el código generado no modifica los módulos marcados como 'sin tocar' en Restricciones	Que la restricción era la correcta en primer lugar
Que los nombres, patrones y convenciones siguen lo definido en la spec de sistema	Que la arquitectura elegida es la más adecuada para el problema
Que todos los casos de error definidos en la spec tienen manejo explícito en el código	Que no existen casos de error que la spec no contemplaba
Que la estructura de ficheros sigue la definida en el proyecto	Que el diseño de la spec es técnicamente óptimo

El riesgo de la validación circular

El riesgo principal del /code-review automatizado es la validación circular: la IA genera spec, código y tests del mismo source of truth. Si la spec estaba equivocada, el código y los tests serán consistentes entre sí pero incorrectos respecto al problema real. El agente pasará la revisión porque el código cumple la spec, aunque la spec no capturaba bien el requisito.

La solución: los criterios de aceptación deben ser escritos por quien conoce el negocio (PM, product owner) y revisados por un humano antes de que el agente ejecute. El /code-review del agente es la primera línea de defensa técnica, no la única.

La estructura de ficheros y cómo escala

OpenSpec organiza el repositorio de forma que la spec sea un ciudadano de primera clase, al mismo nivel que el código:

```

openspec/
├── specs/
│   └── <domain>/
│       └── spec.md
├── changes/
│   ├── <change-name>/
│   │   ├── proposal.md
│   │   ├── design.md
│   │   └── tasks.md
│   └── specs/
│       ├── <domain>/
│       └── spec.md
└── config.yaml

```

Source of truth: el comportamiento del sistema

Una carpeta por cada cambio propuesto

Delta specs: solo lo que cambia

Configuración del proyecto (opcional)

La separación entre **specs/** y **changes/** es la clave. **specs/** contiene el estado actual del sistema —la fuente de verdad permanente. **changes/** contiene los cambios propuestos —uno por feature, con su propia proposal, diseño y tareas. Cuando una feature se archiva, su delta se integra en **specs/** y **changes/** queda limpio para el siguiente ciclo.

Sobre escala en equipos grandes: en un monorepo con múltiples equipos y features concurrentes, estas son las convenciones adicionales que evitan el caos:

- **Naming de changes:** prefijo de ticket + descripción corta. Ejemplo: PROJ-1234-candidate-filters. Facilita la búsqueda y el ownership.
- **TTL de proposals activas:** si una feature lleva más de dos semanas en changes/ sin archivar, es deuda. Configurar alertas en CI para proposals obsoletas.
- **Ownership explícito:** cada carpeta en changes/ tiene un OWNER.md con el nombre del developer responsable y la fecha de inicio del ciclo.
- **Git worktrees por ticket:** cada rama de trabajo tiene su propio contexto activo sin interferir con el resto del equipo. *Una spec activa = un worktree = una rama.*

Lo que genera el ciclo completo automatizado

Cuando el ciclo funciona bien, cada feature produce automáticamente los siguientes artefactos sin que el developer tenga que pedirlos en cada sesión. El sistema lo hace porque el contrato lo define:

- **Tests unitarios** que cubren los criterios de aceptación de la spec, incluyendo los casos de error definidos explícitamente.
- **Tests E2E** que navegan el producto verificando el comportamiento real desde la perspectiva del usuario, alineados con los escenarios de la spec.
- **Pull Request** con descripción estructurada que referencia la spec y los criterios de aceptación verificados.
- **Primera code review automática** con issues detectados antes de que llegue al revisor humano. El revisor llega a un PR ya filtrado.
- **Documentación actualizada** que refleja los cambios realizados, las decisiones técnicas tomadas y los ADRs correspondientes.
- **Análisis preventivo de seguridad** sobre el código generado, señalando patrones de riesgo definidos en la spec de sistema.

Todo esto sin overhead adicional en cada sesión. La inversión está en construir la spec bien. El retorno es que el resto del ciclo se ejecuta con consistencia predecible.

Cuatro prácticas que lo hacen funcionar



Empieza siempre por el contrato.
Antes de pedir nada a la IA, define inputs, outputs, estados y reglas de negocio. Aunque sea en versión mínima, que quede por escrito y versionado.



Diseña para el error, no solo para el éxito.
No basta con describir el happy path. La IA necesita saber cómo manejar errores, límites y estados inválidos. Una spec sin casos de error es una spec a medias.



Usa ejemplos concretos.
No solo reglas de formato o convenciones de nombres: incluye ejemplos de payloads, respuestas y casos de test completos. Ayudan tanto al humano que revisa como al agente que ejecuta.



Trata la spec como código. No es un documento temporal que se pierde en una carpeta. Es parte del sistema. Te da la trazabilidad que hoy la mayoría de equipos no tiene con solo los tickets de Jira: cada commit se puede trazar hasta el requisito que lo originó.

Frameworks SDD: por dónde empezar

SDD se puede aplicar con cualquier copiloto. Lo importante al elegir un framework —o diseñar el tuyo— es que resuelva cuatro cosas: facilidad de arranque, buenas prácticas de manejo del contexto, gestión de cambios en features existentes e integración con tu IDE.

Contenido en vídeo complementario:

Veamos cómo Spec-Driven Development (SDD) permite tener un flujo de trabajo con estándares de alta calidad (incluyendo tests y documentación automática) y qué frameworks puedes teniendo en cuenta sus ventajas, el tipo de proyecto y la complejidad del contexto técnico: <https://youtu.be/ov9GYLPMre8>



Las opciones actuales más relevantes:

- SpecKit — bueno para proyectos greenfield. [\[github.com/github/spec-kit\]](https://github.com/github/spec-kit)
- BMAD — muy potente, curva de arranque alta. Para equipos con madurez en el proceso. [\[github.com/bmad-code-org/BMAD-METHOD\]](https://github.com/bmad-code-org/BMAD-METHOD)
- OpenSpec 🌟 — funciona para proyectos nuevos y existentes. Recomendación LIDR para empezar. [\[github.com/Fission-AI/OpenSpec\]](https://github.com/Fission-AI/OpenSpec)
- ai-specs LIDR — punto de partida si prefieres construir el tuyo. [\[github.com/LIDR-academy/ai-specs\]](https://github.com/LIDR-academy/ai-specs)

Contenido en vídeo complementario:

Una sesión 100% práctica donde construimos una feature completa en vivo aplicando SDD en un proyecto real: de la especificación al código, tests y documentación, sin caos ni rework. <https://eventos.lidr.es/grabacion-workshop-spec-driven-development>

🔗 Repositorio de código
→ github.com/Fission-AI/OpenSpec



08. Casos de éxito. Adopción IA en equipos de desarrollo

La metodología funciona en papel. Estos equipos la tienen funcionando en producción. Tres empresas, tres contextos distintos — una scaleup fintech, una empresa de soluciones para seguros y movilidad, y una de las plataformas de empleo más grandes de España. El denominador común: **la IA como capacidad de equipo, no como herramienta individual.**



E-commerce y fintech · América Latina · +18 países

El problema

A pesar de tener acceso a Copilot y ChatGPT, solo el 30% del equipo usaba IA de forma activa. Sin buenas prácticas compartidas ni métricas de impacto, cada developer improvisaba su propio método. Los líderes de proyecto quedaban fuera del ciclo técnico.

Lo que cambió

Oscar Urrego (Senior Engineering Manager) implementó AI4Devs con acompañamiento 1 a 1. El foco era estandarizar el uso de IA en cada fase del ciclo — código, code reviews y testing — e incorporar a los líderes de proyecto como participantes activos generando tests con IA.

30 % → 92 %
Adopción activa de IA

30 %
Código generado con IA

30 %
Code reviews asistidas

"Lo más valioso de AI4Devs fue su enfoque práctico. Desde el primer día pude aplicar lo aprendido con mi equipo. El programa no solo me dio respuestas, sino las preguntas correctas para seguir innovando en el uso de IA. Para mí la clave está en elegir capacitaciones que transformen la forma de trabajar, no solo que entreguen teoría."

— Oscar Urrego, Senior Engineering Manager, Mercado Libre



Soluciones tech para seguros y movilidad · Scaleup

El problema

Resistencia al cambio en equipos técnicos, cuellos de botella manuales en code reviews y documentación, y ausencia de una metodología clara para medir el impacto real de la IA. Los desafíos no eran técnicos: eran estratégicos.

Lo que cambió

El CTO y los Heads of Engineering de Bdeo formaron a los tres líderes estratégicos del equipo en AI4Devs. Adoptaron Cursor como herramienta principal, establecieron métricas DORA como estándar de medición y crearon agentes internos para automatizar procesos críticos — incluyendo consultas sobre certificaciones ISO.

+16 %
Velocidad de desarrollo

-25 %
Errores en producción

-30 %
De tiempo en code reviews

"La IA no es solo tecnología; es cambio cultural. Medir, comunicar y acompañar al equipo es tan importante como elegir las herramientas."

— Rafael Castillo, VP of Technology, Bdeo

InfoJobs

Plataforma de empleo · +11M candidatos · Enterprise

El problema

Adopción desigual: los juniors usaban IA, los seniors la evitaban. Miedo a la "caja negra" en sistemas críticos. Ausencia de criterios compartidos que dieran confianza al equipo para trabajar con IA en producción.

Lo que cambió

Tomás Casquero (Engineering Manager) implementó AI4Devs B2B con enfoque en el ciclo completo, no solo en generación de código. El equipo creó MCPs conectados a Jira y GitHub para hacer cumplir estándares automáticamente. Un desarrollador construyó un asistente basado en PRs históricos para distribuir conocimiento sin depender de personas concretas.

1 día
Documentar microservicio
(antes: 1 semana)

MCPs
Estándares aplicados
automáticamente

0 dependencias
Conocimiento del equipo
distribuido

"Destacaría la naturaleza práctica de la formación. Tras una sesión teórica inicial esencial para alinear el entendimiento de todos, las sesiones posteriores fueron eminentemente prácticas, centradas en escenarios reales para distintos perfiles (backend, iOS, frontend)".

— Tomás Casquero, Engineering Manager, InfoJobs

**¿Tu equipo
está en este
punto?**

AI4Devs B2B está diseñado para llevar la adopción de IA al nivel de equipo con metodología y métricas. Mismo programa. Adaptado a tu stack, tu contexto y tus proyectos.

Solicita una llamada → business@lidr.co · lidr.co/empresas

09. AI Assessment 2026: descubre tu nivel

Esto no es nada nuevo: la IA no te va a reemplazar. Te reemplazará alguien que use IA. El primer paso es saber dónde estás. El AI Assessment de LIDR es una evaluación gratuita que mide tu nivel real de adopción de IA y te da un diagnóstico personalizado con comparativa de mercado y recomendaciones de mejora concretas.

Qué obtendrás al completar la evaluación

 Diagnóstico en 6 dimensiones Uso individual, uso colectivo, liderazgo técnico, contexto operativo, mejora continua y resultados.	 Recomendaciones accionables Plan de mejora para las áreas con más oportunidad, con recursos curados de la academia LIDR.
 Comparativa con el mercado Compara tu perfil con profesionales similares. Descubre si estás por encima o por debajo del promedio.	 Resultados inmediatos y gratis Sin registro obligatorio, sin tarjeta de crédito. Recibe tus resultados al instante.

Elige tu evaluación

Cada assessment mide un área específica. Empieza por la que más te interese.

AI-Assisted Development RECOMENDADO PARA EMPEZAR Evalúa tu madurez en el uso de herramientas de IA en desarrollo. ⌚ 3-5 min · 📄 12 preguntas · ai-assessment.lidr.co/es/ai-assisted	AI Maturity Assessment AVANZADO Evalúa la madurez de adopción de IA de tu equipo en 6 dimensiones. ⌚ 8-13 min · 📄 30 preguntas · ai-assessment.lidr.co/es/ai-maturity
---	---

Dato relevante:

Según el Pluralsight 2026 Tech Forecast, las organizaciones que miden su adopción de IA con criterios claros y tienen un plan de mejora estructurado tienen 3.4x más probabilidades de reportar ROI positivo en los 6 meses siguientes.



Completa una evaluación gratuita y recibe acceso a contenido exclusivo de la academia LIDR.
Empieza gratis → ai-assessment.lidr.co/es

10. ¿Quieres convertirte en Agentic Engineer?

Si has leído este playbook, ya lo viste: la IA no falla por "alucinar" tanto como falla por falta de *definición, contexto y validación*. Ser AI-Powered Developer no va de saber prompts: va de convertir la IA en un sistema repetible y escalable sin perder el control.

El Máster AI4Devs es el programa que te lleva de usar copilotos de forma aislada a dominar el ciclo completo de desarrollo con IA — desde el análisis y la planificación hasta el testing, el frontend y el backend. Con mentores en activo, proyectos reales y una comunidad de profesionales tech en España y Latinoamérica.

Para quién es AI4Devs

Developers mid-senior

Quieren entrar al mercado con una ventaja diferencial real y un onboarding acelerado.

Tech Leads y Product Owners técnicos

Quieren multiplicar su impacto y convertirse en el referente de IA en su equipo.

Engineering managers, CEOs y CTOs

Necesitan llevar la adopción de IA al nivel de equipo con método, métricas y estándares.

Qué incluye

32 horas Sesiones en vivo	36 horas 30 prácticas + 6 labs opcionales	Próxima edición Inscripciones abiertas Plazas limitadas
SESIONES EN VIVO LOS MARTES · 17:30–19:30 hs CET 11:30–13:30 hs COT		
 Clases presenciales online Sesiones en directo con demos prácticas y casos de estudio reales. Grabaciones por 12 meses.	 Programación práctica No es teoría: código real desde el primer día. 30 horas construyendo funcionalidades y lanzando tu producto.	
 Q&A con mentores TOP Espacio seguro para traer tus problemas y desafíos actuales con profesionales con experiencia para ayudarte.	 Contenido complementario Acceso a contenido programado y recursos seleccionados para profundizar en cada tema a tu ritmo.	
 Certificado final Al completar el Máster obtienes un certificado validado por LIDR que acredita tus competencias.	 Comunidad exclusiva Red privada de expertos, líderes y oportunidades. Eventos, novedades y empleos TOP para acelerar tu carrera.	

Mentores

Líderes tech en activo con más de 10-20 años construyendo y escalando productos en empresas top. Combinan experiencia hands-on en desarrollo, IA y automatización con una orientación muy práctica: te enseñan exactamente cómo aplicar lo que saben en tu código, tus proyectos y tu carrera del día a día.



Álvaro Moya

CTO, mentor IA y fundador de LIDR
Más de 12 años liderando equipos tech. Acompaña a developers y startups en la adopción real de IA. Ha formado a más de 2.000 ingenieros.



Antonio Pérez

CIO en Secture Labs
Más de 20 años en desarrollo de software. Lidera proyectos complejos y explora arquitecturas IA aplicadas en entornos reales.



Miguel Camacho

CTO, formador y experto en IA
Ingeniero por UPM e IIT Chicago. Especialista en Big Data y Machine Learning. Apasionado del liderazgo y el desarrollo hands-on.



Eduardo Hilario

CTO en IA for Transport
Ha liderado proyectos de IA en logística en entornos complejos. Visión aplicada y foco en resultados desde el primer día.



Leifer Méndez

Senior Software Engineer
Software Engineer y formador con +40K alumnos. Enseña a aplicar todo lo aprendido desde cero en proyectos propios.



Pedro Ávila

Software Engineering Manager
Director de Ingeniería con amplia experiencia en IA. Ha implementado múltiples soluciones de automatización y optimización de procesos.

Ya confían en nosotros

Organizaciones TOP como Mercado Libre, Factorial, Bdeo, Fever, Telefónica, Adevinta, SEAT:Code, Wallapop, Santander, Globant, Rappi, Latam Airlines, Idealista, Roche y muchas más han formado a sus equipos con LIDR



Lo que dicen los developers que ya pasaron por el programa

"Este máster ha sido una de las mejores decisiones de mi carrera. Me dio la base y las prácticas para usar IA en desarrollo de software de forma efectiva y enfocada a negocio. Pude crear un MVP funcional en solo un día aplicando prompting, testing y Cursor IDE."

— Erik Rodríguez, Full Stack AI Developer

"Este máster fue clave para integrar IA en todo mi flujo de trabajo como developer. Aprendí desde el uso de LLMs y prompt engineering hasta la aplicación práctica en testing, reuniones y desarrollo fullstack. Multipliqué mi velocidad de desarrollo y la calidad de mis entregables."

— Abel Acuña, AI-Powered Developer

"Completar AI4Devs fue una experiencia transformadora. Aprendí no solo herramientas y técnicas, sino que formé nueva forma de pensar el desarrollo con IA. El programa me empujó a salir de lo operativo y adoptar una mirada más estratégica, colaborativa y orientada al impacto."

— Gerardo Hernández, Senior AI Engineer

"Después de 5 ediciones previas, puedo afirmar con orgullo que estoy más satisfecho con esta experiencia. La calidad del contenido es extraordinaria. AI4Devs no es solo el Máster con más rigor: es una formación fundamental en mi vida como profesional."

— Leonardo Benítez, Responsable Desarrollo Software

"AI4Devs marcó un antes y un después en mi forma de aprender, pensar y construir tecnología. El nivel técnico, de comunidad y la intensidad del programa me engancharon desde el primer día. Aprendí a integrar IA en mis proyectos de forma natural, automatizando tareas y ampliando la calidad del código."

— Leandro Alberto García, Líder Técnico

"AI4Devs ha sido mucho más que un Máster para mí. Ha sido un depósito de conocimientos, recursos y técnicas sobre visión: técnicas de negocio, producto y desarrollo. Comunicación, resolución de conflictos, mentoring. Es y será un montón de temas más. ¿Qué más pedir?"

— Daniel Pastrana Rubio, Engineering Manager

Garantía de devolución y facilidades de pago



30 días de garantía. 0 riesgo

Garantía de devolución del dinero de 30 días desde que accedes al máster. Si no funciona o no es para ti, tienes derecho a solicitar la devolución. Sin preguntas.



Facilidades de pago

Opciones de financiación del coste en cuotas sin intereses. Reserva tu entrevista de admisión y nuestro equipo te dará todos los detalles.



Bonifica con tu empresa

En España el curso es parcialmente bonificable por FUNDAE (hasta 700€). Podemos ayudarte a que tu empresa dedique un presupuesto a la formación.

Proceso de inscripción

El acceso a AI4Devs es mediante entrevista de admisión. Queremos asegurarnos de que el programa es el adecuado para ti y de que puedes sacarle el máximo partido desde el primer día.

1

Solicita tu entrevista de admisión

Rellena el formulario en lidr.co/ia-devs. Nuestro equipo se pondrá en contacto contigo para agendar la llamada.

2

Entrevista de admisión

Una llamada de 30 minutos con el equipo de LIDR. Evaluamos tu perfil, resolvemos tus dudas y confirmamos que el programa encaja con tus objetivos.

3

Confirmación y acceso

Si la admisión es positiva, recibes el acceso a la plataforma, la comunidad y los materiales de bienvenida antes del inicio de la edición.



Solicita tu entrevista de admisión → lidr.co/ia-devs

*"Los developers predijeron que la IA reduciría su tiempo un 24%. Al terminar, seguían creyendo que les había hecho un 20% más rápidos. **La realidad era exactamente la opuesta.**"*

— METR Lab, 2025



Empezaste este playbook con una paradoja.

La IA te acelera... hasta que revisas el PR.

Si has llegado hasta aquí, ya tienes el método para que eso no vuelva a pasar.

¿Dónde estás ahora?

Evalúa tu nivel real de adopción de IA en 6 dimensiones.

Gratis. Resultados inmediatos.

EMPIEZA TU AI ASSESSMENT →

ai-assessment.lidr.co/es